



UNIT 22: APPLICATION DEVELOPMENT

(MATH RUSH 3D REPORT)

VINCENT ESCANDALLO CASTILLO

TJ55730

REPORT SUBMITTED FOR THE

PEARSON BTEC HIGHER NATIONAL DIPLOMA IN COMPUTING

COSMOPOLITAN COLLEGE OF COMMERCE AND TECHNOLOGY

Report scope and evidence status

Evidence status: This report addresses the supplied project guideline and assignment brief using the inspected Math Rush 3D implementation. Missing peer-review, device-performance and administration evidence is stated explicitly and is not presented as completed work.

Revision record

Version	Date	Purpose	Status
0.1	21 July 2026	Initial report based on currently supplied guidelines	Draft
0.2	29 July 2026	Reformatted to the requested Times New Roman academic style and expanded traceability, performance and ethics sections	Revised draft
1.0	4 August 2026	Final report review, current build verification and diagram layout correction	Final report

Table of contents

Report scope and evidence status	2
Revision record.....	2
Table of contents.....	3
Executive summary	5
1. Project context and scope	7
1.1 Scenario	7
1.2 Aim	7
1.3 Objectives	7
1.4 Success measures.....	7
1.5 Scope boundaries	8
2. Software Design Document.....	9
2.1 Problem definition	9
2.2 Stakeholder analysis	9
2.3 User requirements	10
2.4 System requirements.....	11
Functional requirements	11
Non-functional requirements	11
2.5 Assumptions and constraints	12
2.6 Risk review and mitigation.....	12
2.7 Use cases.....	13
2.8 Data design	14
2.9 Three-tier architecture	15
2.10 Data-flow diagrams	16
2.11 Gameplay flowchart.....	18

2.12 Interface wireframes	19
2.13 Requirements traceability	19
3. Development technologies and methodology.....	21
3.1 Software and tools used	21
3.2 Alternatives considered	22
3.3 Frontend implementation.....	23
3.4 Backend implementation	23
3.5 Database and SQL	24
3.6 Development methodology.....	24
3.7 Version control and delivery	25
3.8 Data protection, ethics and asset licensing	26
4. Testing and user acceptance	27
4.1 Testing strategy.....	27
4.2 Current automated evidence	27
4.3 Test data and environments	27
4.4 Peer tester plan.....	28
4.5 User acceptance checklist.....	28
4.6 Performance evaluation plan	29
4.7 Defect and feedback handling	30
5. Current assignment coverage and gaps.....	32
5.1 Mandatory feature gap analysis	33
5.2 Prioritised next stage.....	34
6. Conclusion	35
References.....	36
Appendix A. Evidence still to insert.....	37
Appendix B. Project technology snapshot.....	38

Executive summary

Math Rush 3D is a mobile-first three-dimensional crowd-running game that combines fast arcade play with short arithmetic decisions. The project responds to a practical engagement problem: learners often experience basic arithmetic practice as repetitive and disconnected from immediate consequences. In Math Rush 3D, addition, subtraction, multiplication and division are placed directly in the player's route. Correct or advantageous decisions visibly change the size of the running crowd, which then determines whether the player can survive obstacles, defeat an end-level boss and reach a higher bonus multiplier.

The implemented product is delivered as both a responsive web application and an Android package. The presentation tier uses React, TypeScript, Tailwind CSS, Three.js and React Three Fiber. Zustand coordinates interface and run state without forcing frame-by-frame React updates. The application tier is currently an Express REST API written in TypeScript, with Zod validation, rate limiting, security headers and session-based authentication. PostgreSQL stores accounts, settings, progression, run summaries, gate choices, obstacle events, skins, achievements and leaderboard data. Production services use Render and Neon, while local development can use Docker.

A typical session begins with registration or login, followed by difficulty and level selection. The player steers a clustered crowd through ten pairs of arithmetic gates and difficulty-specific obstacles. Enemy crowds and hazards remove individual runners. Immediately before the boss, the player stops a timing meter to earn a crowd boost. The boss battle consumes crowd members according to the level balance. If successful, the survivors enter a multiplier lane, where each bonus stage exchanges crowd members for bonus points until the crowd is exhausted. The result screen awards a score, coins, stars, level progression and a next-level option.

The current implementation demonstrates a substantial functional baseline: four difficulty bands with five levels each, responsive controls, GPU-instanced crowds, collision tests based on movement-plane crossing, a balanced boss calculation, skins, persistent accounts and an online leaderboard. On 4 August 2026, all 30 automated tests passed, the browser and server TypeScript checks completed successfully, and Vite produced a production build. These checks support the internal consistency of the balancing, formation, level-progression, speed and boss-meter utilities, but they do not replace device testing, accessibility review, penetration testing or user acceptance testing.

Several assignment-compliance items remain open. The brief lists PHP, Python or Java as expected backend technologies and permits Node.js only as middleware or API glue; therefore, the current Node.js/Express backend requires explicit tutor approval or replacement by an accepted service. Role-based access control, an administrator interface, full core-entity CRUD, a formal audit trail, a feedback store and completed peer-review evidence are also not yet implemented. These gaps are documented rather than hidden so that the next development iteration can be planned against the marking criteria.

The recommended next stage is to obtain written confirmation about backend-language compliance, then implement an administrator role and management workflow, complete remaining CRUD/search/report functions, collect peer feedback using the UAT checklist, and add dated screenshots and GitHub evidence. The design and research in this report provide a traceable starting point for those activities and can be expanded when the remaining guideline is supplied.

1. Project context and scope

1.1 Scenario

The project is framed as a GameForge Studios hackathon entry. The challenge is to create a browser-based game that addresses a real-world problem while demonstrating application-development practice, relational data management and evidence-led evaluation. Math Rush 3D interprets the real-world problem as low engagement with repeated mental-arithmetic practice, particularly when exercises provide little agency or visual feedback.

1.2 Aim

The aim is to create a responsive, replayable arithmetic game in which a player's mathematical decisions and movement skill jointly determine success, while a secure data service records progress, settings and competitive results across web and Android clients.

1.3 Objectives

- Deliver a complete start-to-finish game loop with gates, hazards, enemy crowds, a boss and a finite bonus ending.
- Scale arithmetic content, obstacles and movement speed across four difficulty bands and five levels per band.
- Maintain fair level balance so that an optimal route remains achievable while poor choices have meaningful consequences.
- Provide account registration, login, saved progression, skins, statistics and leaderboards through PostgreSQL.
- Support touch, mouse and keyboard interaction on responsive web layouts and Android devices.
- Collect evidence through automated checks, manual tests, peer UAT and GitHub history.

1.4 Success measures

Measure	Target	Evidence method
Mathematical correctness	All generated expressions return the displayed integer result and update the crowd once.	Automated gate tests plus sampled gameplay evidence.

Measure	Target	Evidence method
Level fairness	An optimal route can defeat the boss in every published difficulty and level.	Balance tests and documented perfect-route play tests.
Mobile usability	Core controls and authentication remain usable at 360x640 and larger supported viewports.	Responsive browser matrix and Android UAT.
Runtime performance	Gameplay should normally remain at or above 45 FPS on the selected mid-range Android test device.	Recorded frame-rate and frame-time test runs.
Persistence reliability	A completed authenticated run appears once in progress and leaderboard records after reconnection.	API/database integration test and offline queue test.
Security baseline	Invalid input is rejected, passwords are never stored in plain text and unauthorised admin access is denied.	Validation, database inspection and authorisation tests.

1.5 Scope boundaries

The current scope includes a single-player crowd runner, arithmetic gates, local 3D simulation, account-backed persistence and competitive rankings. Real-time multiplayer, payments, advertisements, social messaging, user-generated levels and an iOS package are excluded. The administrator role and formal feedback workflow are required by the brief but are identified as planned work because they are not present in the inspected repository.

2. Software Design Document

2.1 Problem definition

Conventional arithmetic worksheets can test correctness but rarely sustain voluntary repetition. They also separate the calculation from a meaningful consequence. Math Rush 3D turns arithmetic into an immediate spatial decision: the player reads two expressions, steers towards one answer and sees the crowd change. The design must avoid becoming a guessing game, however. Symbols must remain legible on small screens, level generation must not create impossible encounters, and movement skill must reduce casualties without allowing all hazards to be ignored.

2.2 Stakeholder analysis

Stakeholder	Need or interest	Influence	Design response
Learner / player	Clear rules, fair challenge, responsive steering, visible rewards and saved progress.	High	Difficulty progression, readable gates, touch controls, feedback and next-level flow.
Lecturer / assessor	Evidence that the solution satisfies the brief and is evaluated honestly.	High	Traceable requirements, diagrams, testing records, references and explicit compliance gaps.
Game developer	Maintainable modules, reproducible builds and measurable defects.	High	TypeScript, separated game systems, shared rules, tests and GitHub workflow.
Administrator	Manage users/content and inspect reports.	Medium	Planned RBAC, administration dashboard, audit log and CRUD endpoints.
Hosting/data base provider	Secure, efficient and correctly configured workloads.	Medium	Connection pooling, environment variables, rate limits and health checks.
Parent/educator	Age-appropriate play and evidence of mathematical value.	Low/medium	No chat or payments; arithmetic decisions and progression statistics.

2.3 User requirements

ID	Requirement	Acceptance measure	Status
UR-01	A visitor can register and an existing player can log in.	Valid credentials create a persisted session; invalid input receives a clear error.	Implemented
UR-02	A player can choose an unlocked difficulty and level.	Locked stages are visibly unavailable and completing a stage unlocks the next valid stage.	Implemented
UR-03	A player can steer by touch, mouse or keyboard.	The crowd moves laterally within track boundaries without page scrolling during play.	Implemented
UR-04	Gate content contains difficulty-appropriate arithmetic expressions.	Addition, subtraction, multiplication and division appear at suitable complexity and produce integer outcomes.	Implemented
UR-05	Individual crowd members can be removed by hazards.	A hit reduces the crowd by the calculated amount and produces visible feedback.	Implemented
UR-06	The level has a boss and a finite bonus ending.	A successful boss fight enters the multiplier lane and finishes at a visible end structure.	Implemented
UR-07	A player can pause, restart, replay or continue to the next level.	Controls change the current run without requiring a browser refresh.	Implemented
UR-08	Progress, scores, settings and skins remain available after sign-in on another client.	Server-backed profile values reload after authentication.	Implemented
UR-09	Players can view rankings and personal statistics.	Leaderboard and statistics panels retrieve current stored data.	Implemented / verify UAT
UR-10	An administrator can manage users, game content and reports.	Authorised admin screens support protected CRUD and reporting actions.	Planned

2.4 System requirements

Functional requirements

- FR-01: generate exactly ten arithmetic gate pairs for a standard level and retain the chosen and optimal values for run evidence.
- FR-02: calculate obstacle, enemy and boss outcomes from a deterministic level definition shared by all scene systems.
- FR-03: validate registration, login, player updates and run submissions at the API boundary.
- FR-04: use parameterised SQL to create and retrieve account, progression, run, event, skin and leaderboard records.
- FR-05: prevent duplicate run submission by treating the client run identifier as unique.
- FR-06: support offline/local play and queue a completed run for later synchronisation when the API is unavailable.
- FR-07: calculate rewards using a shared rule module so browser and API calculations cannot silently diverge.
- FR-08: provide protected administrator CRUD, RBAC, audit reporting, feedback collection and search/filter functions (planned requirement).

Non-functional requirements

- NFR-01 Performance: target smooth play on mainstream mobile devices by using instanced crowds, capped pixel ratio, reduced-effects settings and ref-based frame data.
- NFR-02 Usability: maintain readable arithmetic, large touch targets, safe-area spacing and scrollable authentication forms on small screens.
- NFR-03 Reliability: recover gracefully when the API is sleeping or temporarily unreachable; avoid losing a locally completed run.
- NFR-04 Security: hash passwords, store session tokens as hashes, use HTTPS in production, apply security headers, rate-limit authentication and keep secrets outside the repository.
- NFR-05 Maintainability: separate scene systems, pure game rules, API routes, validation, database access and shared reward logic.
- NFR-06 Compatibility: support current Chromium-based browsers and Android 7.0/API 24 or later through Capacitor.
- NFR-07 Accessibility: preserve keyboard operation outside gesture-only gameplay, meaningful labels, sufficient contrast and reduced-effects options; formal WCAG testing remains pending.

2.5 Assumptions and constraints

The design assumes an internet connection is available for account synchronisation, but the core run remains local and playable during an API outage. Render's free service may cold-start, so the first request can be slower than subsequent requests. The 3D scene is constrained by mobile GPU and memory limits, making instancing and bounded effects more important than photorealistic assets. The largest academic constraint is that the current Express API may not satisfy the brief's prescribed backend-language list; this must be resolved with the tutor before final submission.

2.6 Risk review and mitigation

Risk	Likelihood	Impact	Mitigation / contingency	Owner
Generated level is impossible even with optimal play	Medium	High	Balance from the same level definition; automated game-balance tests; play-test every difficulty/level seed policy.	Developer
Frame-rate drop with large crowds	Medium	High	GPU instancing, reusable geometry/materials, limited pixel ratio, reduced-effects mode and device profiling.	Developer
API cold start is interpreted as login failure	High	Medium	Visible connecting state, retry/backoff, health check and clearer timeout message; consider paid/always-on hosting later.	Developer
Credential or session compromise	Low/medium	High	bcrypt work factor, token hashing, secure HTTP-only cookie, short-scoped bearer storage, rate limits, HTTPS and session expiry.	Developer
Leaderboard score is manipulated	Medium	High	Server recalculation from submitted events, stricter plausibility validation, signed build/version policy and anomaly reports.	Developer/Admin
Database unavailable or migration fails	Low/medium	High	Connection pool health checks, repeatable migrations, Neon backups/branching and documented restore procedure.	Developer
Mobile form clips controls	Medium	Medium	Dynamic viewport units, safe-area padding, vertical scrolling and tests on short/narrow devices.	Developer/Tester
Third-party model licence	Low	High	Use CC0 assets where possible, retain attribution file and store source/licence	Developer

Risk	Likelihood	Impact	Mitigation / contingency	Owner
is unclear			metadata with each asset.	
Backend stack does not meet brief	High	High	Obtain written tutor approval or reimplement the service in Python, Java or PHP before submission.	Student/ Tutor
Peer feedback is unavailable before deadline	Medium	Medium	Book named testers early, use the UAT checklist, record device/browser and preserve dated evidence.	Student

2.7 Use cases

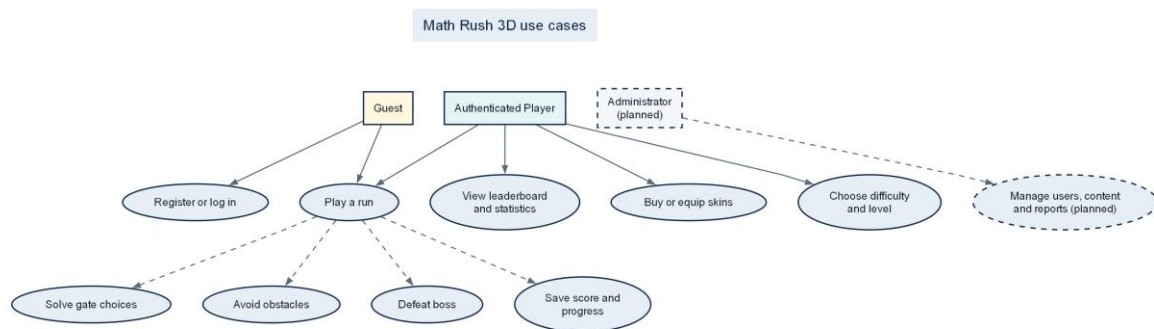
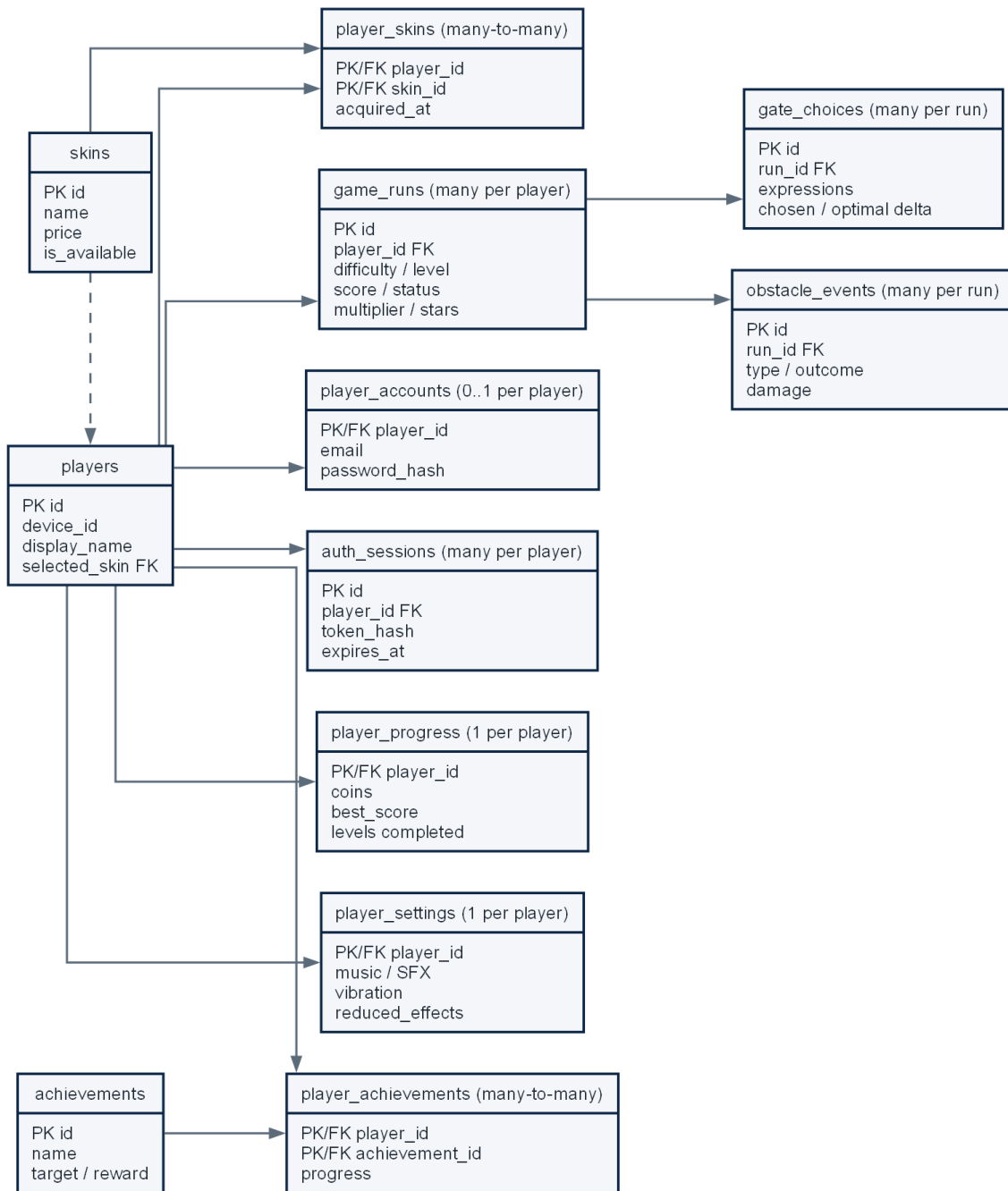


Figure 1. Use-case model; dashed administration functions are planned.

2.8 Data design*Figure 2. Current PostgreSQL entity-relationship model.*

The schema normalises identity, credentials, mutable progression, settings and repeatable run/event records. Foreign keys and cascading deletes protect referential integrity, while checks constrain counts, difficulty values, outcomes and score-related fields. A leaderboard view joins player identity with

progress data. The design currently lacks roles, permissions, administrative audit records and feedback entities; these should be added without overloading the existing gameplay event tables.

2.9 Three-tier architecture

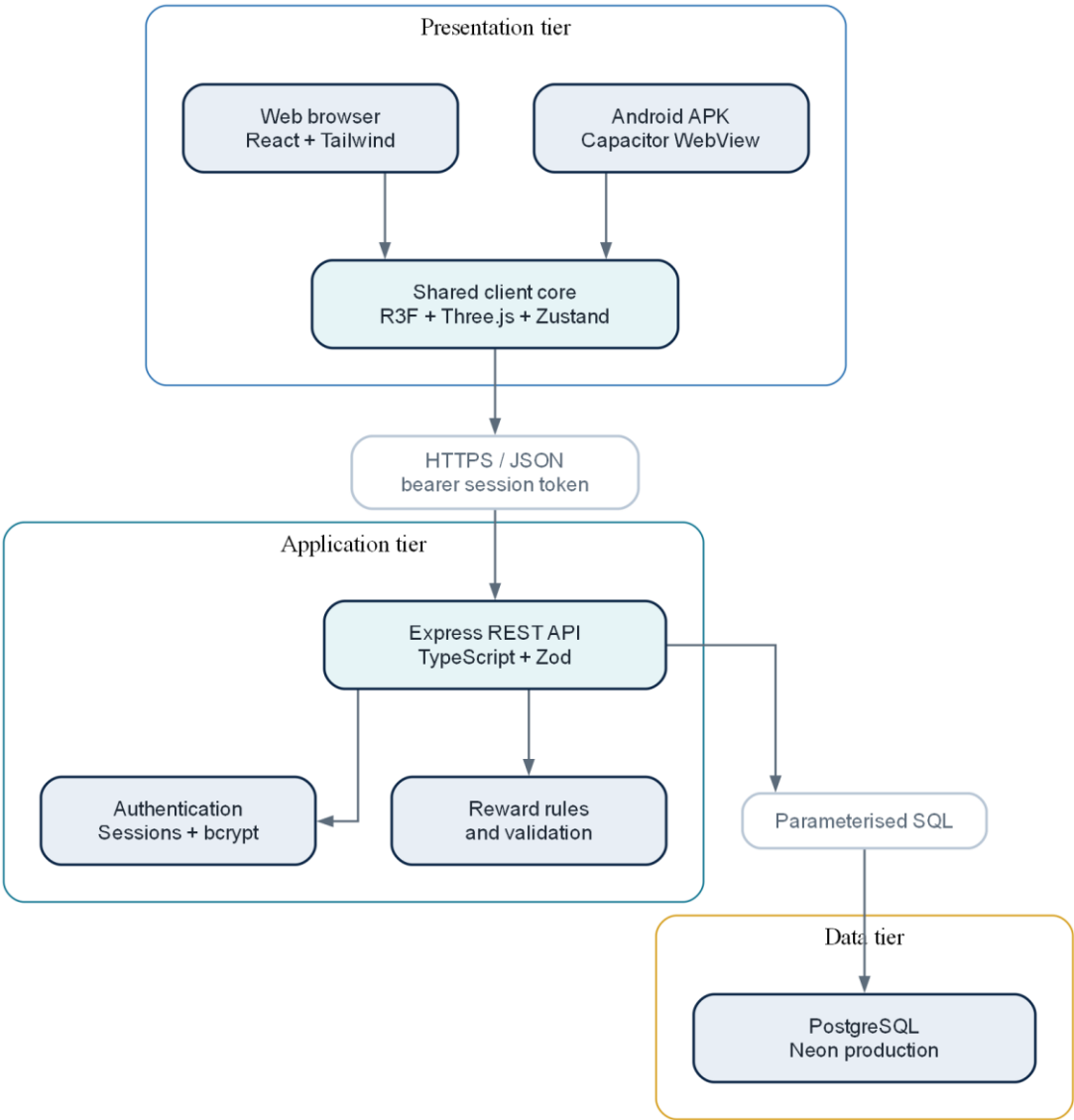


Figure 3. Three-tier deployment and runtime architecture.

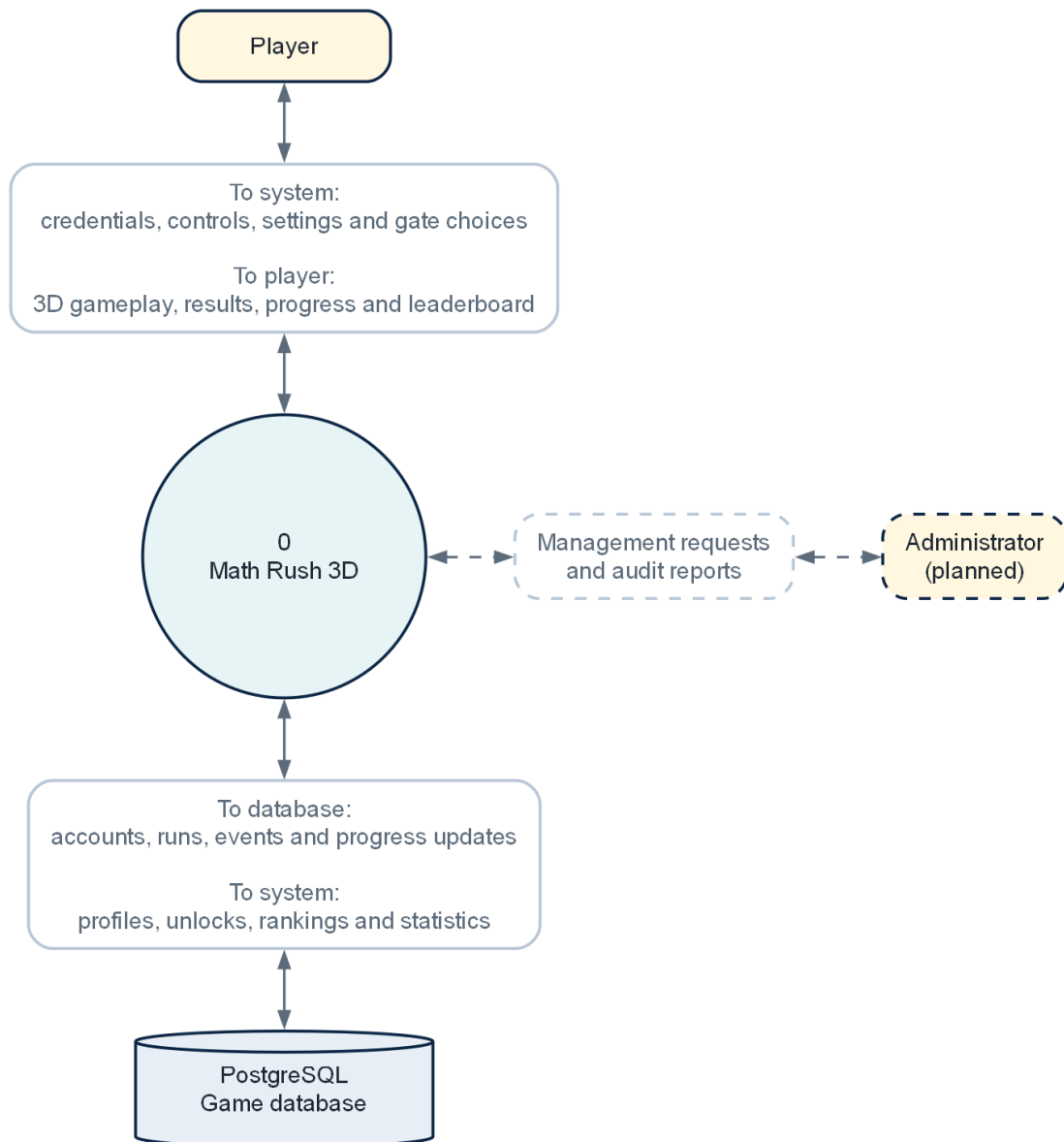
2.10 Data-flow diagrams

Figure 4. Level 0 context data-flow diagram.

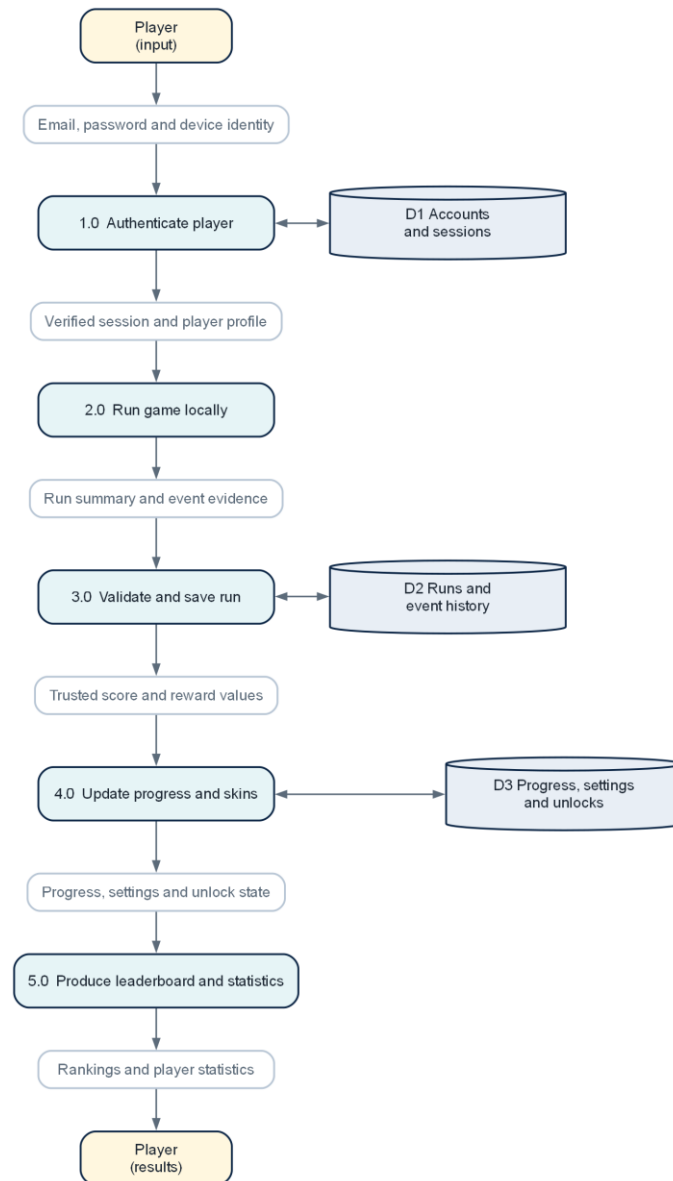
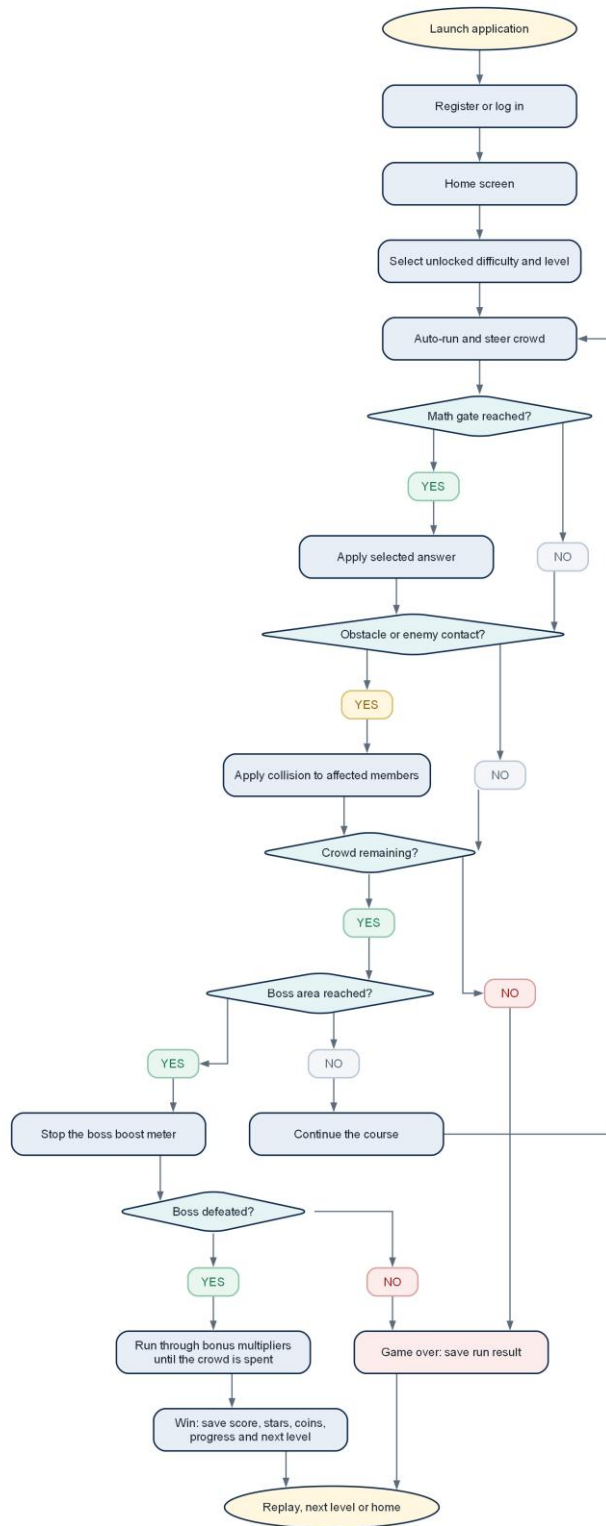
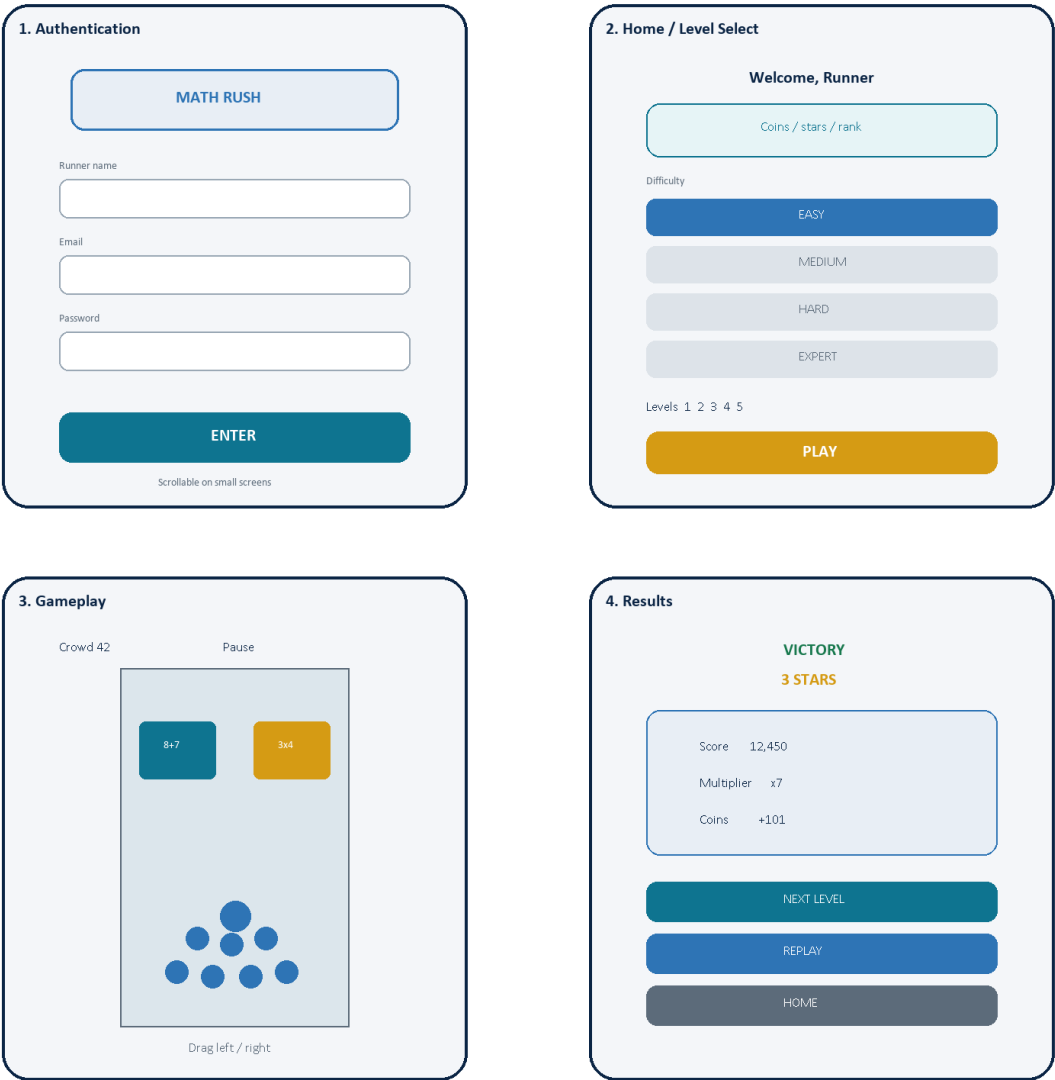


Figure 5. Level 1 data-flow diagram.

2.11 Gameplay flowchart*Figure 6. End-to-end gameplay flow.*

2.12 Interface wireframes

Math Rush 3D: low-fidelity mobile wireframes



Four principal mobile states are shown at a readable size. Final annotated Figma evidence remains to be supplied.

Figure 7. Low-fidelity mobile wireframes for the four principal interface states.

2.13 Requirements traceability

The traceability matrix links each user requirement to the current implementation area and the evidence still needed. It prevents a feature from being treated as complete only because it appears on screen.

Requirement	Implementation area	Verification	Current position
UR-01 Authentication	AuthScreen, authentication API routes, player_accounts and auth_sessions	Registration, login, invalid input and session-expiry integration tests	Implemented; formal integration evidence pending
UR-02 Progression	HomeScreen, levelProgress utilities and player_progress	Unlock-order unit tests and cross-device profile test	Implemented
UR-03 Steering	useInputController, CrowdRunner and track boundaries	Touch, mouse and keyboard device matrix	Implemented; UAT pending
UR-04 Arithmetic gates	mathGates utilities and GateManager	Expression correctness tests across all difficulties	Implemented
UR-05 Individual casualties	CrowdRuntime, ObstacleManager and obstacle events	Collision and crowd-loss regression tests	Implemented; animation UAT pending
UR-06 Boss and finish	BossManager, boss meter, GameTrack and bonus stages	Perfect-route balance test and end-state UAT	Implemented
UR-07 Run controls	Pause overlay, resetGame, replay and next-level actions	Pause, restart and duplicate-submission checks	Implemented
UR-08 Persistence	Zustand persistence, player API, run queue and PostgreSQL	Offline queue plus second-client login test	Implemented; integration evidence pending
UR-09 Rankings	Leaderboard route, view and statistics modal	Ordering, filtering and privacy tests	Partially verified
UR-10 Administration	No protected administrator module identified	RBAC denial and authorised CRUD tests	Not implemented

3. Development technologies and methodology

3.1 Software and tools used

Area	Selected technology	Purpose and justification
Interface	React 19 + React DOM	Component-based menus and overlays; React's state and composition model suits reusable screens and controls (Meta Platforms, 2026).
Language	TypeScript 6	Static types across browser, server and shared rules reduce mismatched event and API structures (Microsoft, 2026).
Build	Vite 8	Fast development server and optimised production bundling for a modern React application (Vite, 2026).
Styling	Tailwind CSS 4 + project CSS	Responsive utility classes support consistent spacing and breakpoints while custom CSS handles game-specific surfaces (Tailwind Labs, 2026).
3D	Three.js + React Three Fiber + Drei	WebGL rendering with a declarative React scene graph and reusable helpers; R3F is explicitly a React renderer for Three.js (Poimandres, 2026).
State	Zustand	Small central store with selective subscriptions and persistence; frame-critical transforms remain outside reactive state.
API	Node.js + Express 5 + Zod	REST endpoints, middleware, validation and shared TypeScript. Academically, this selection needs tutor approval because of the brief's language restriction.
Database	PostgreSQL 17/18 + pg	Relational constraints, foreign keys, views, transactions and parameterised SQL; PostgreSQL supports complex queries and transactional integrity (PostgreSQL Global Development Group, 2026).
Mobile	Capacitor 8 + Android/Gradle	Packages the web-first application in a native Android container and retains access to native platform APIs (Ionic, 2026).
Testing	Vitest + TypeScript + Oxlint	Fast rule-level regression tests, compile-time checks and static analysis.
Delivery	GitHub + GitHub Actions + Render + Neon	Version history, automated APK workflow, static/API hosting and managed PostgreSQL.

Area	Selected technology	Purpose and justification
Design/evidence	Word, Graphviz and low-fidelity wireframes	Structured report, repeatable technical diagrams and an initial screen plan; editable Figma evidence remains pending.

3.2 Alternatives considered

Decision	Selected	Alternative	Reasoned comparison
3D integration	React Three Fiber	Plain Three.js	Plain Three.js gives direct control and fewer React concepts, but R3F keeps the scene declarative and componentised while still exposing Three.js objects. The current team skill set and React UI favour R3F.
State management	Zustand	React Context / Redux Toolkit	Context is sufficient for low-frequency UI state but can become awkward across many game consumers. Redux offers stronger conventions and tooling but adds ceremony. Zustand is compact; careful selectors are still required.
Server	Express/TypeScript	FastAPI/Python, Spring Boot/Java, Laravel/PHP	Express maximises type sharing and implementation speed. FastAPI would satisfy the brief and provide strong validation/OpenAPI; Spring Boot offers mature structure but higher complexity; Laravel offers rapid CRUD and RBAC patterns. Compliance may outweigh reuse.
Database access	Direct pg SQL	Prisma ORM	Direct SQL makes schema and queries visible for assessment and avoids ORM overhead. Prisma would improve generated types and migration ergonomics but can hide SQL learning and adds a generation layer.
Database	PostgreSQL	MySQL / SQLite	PostgreSQL is required by the brief and supports constraints, views and rich querying. SQLite is attractive offline but unsuitable as the shared production leaderboard store.

Decision	Selected	Alternative	Reasoned comparison
Mobile delivery	Capacitor	React Native / native Android	Capacitor reuses the deployed web client and is appropriate for the current project. React Native or native Android could improve device-specific optimisation but would duplicate rendering and interface work.
Methodology	Agile iterative	Waterfall	The game required repeated balancing and mobile feedback, which benefits from short build-test-review loops. Waterfall offers stable documentation gates but delays gameplay validation until too late.

3.3 Frontend implementation

React components own the screen-level interface, while GameScene creates a level definition and passes it to specialised scene systems. GateManager, ObstacleManager and BossManager respond to the same generated values; CrowdRunner and CrowdRuntime maintain movement and formation state; FollowCamera tracks the action; GameTrack renders the route and finish. React Three Fiber permits these scene responsibilities to remain component-based while the per-frame loop operates on Three.js objects rather than rebuilding ordinary DOM elements (Poimandres, 2026).

The Zustand store models phase, run stage, difficulty, selected level, profile, settings, rewards and event summaries. Long-lived preferences are persisted locally and reconciled with the authenticated profile. Position and animation data that change every frame are kept in refs and runtime objects to avoid excessive component rerendering. The crowd uses instanced drawing so many bodies can share geometry and material data.

Responsive behaviour includes touch dragging, mouse/keyboard steering, safe-area insets, capped device pixel ratio, a reduced-effects option and scrollable authentication content. Capacitor provides the native Android shell while preserving the same React application, consistent with its web-first runtime model (Ionic, 2026).

3.4 Backend implementation

Express exposes health, authentication, player, run and leaderboard routes. Request bodies and parameters are validated with Zod. Helmet configures common security headers, CORS restricts

approved client origins, JSON bodies are limited and authentication endpoints are rate-limited. Passwords are hashed with bcrypt at cost 12. This meets OWASP's stated minimum bcrypt work factor of 10, although OWASP prefers Argon2id for new systems where available (OWASP Foundation, 2026a). Random session tokens are returned to the client but only SHA-256 token hashes are stored in the database, limiting direct reuse after a database leak.

Browser sessions use secure HTTP-only cookies in production, while the Android client uses a bearer token because WebView cookie behaviour differs across origins. Sessions expire after 30 days and can be invalidated. OWASP describes the session identifier as temporarily equivalent to the strongest authentication method, so HTTPS, secure storage, expiry, rotation and logout behaviour must be tested as security controls rather than treated as interface details (OWASP Foundation, 2026b).

Compliance decision required: The assignment brief names PHP, Python or Java for the backend and allows Node.js only as middleware or API glue. The current Express service performs core business and persistence work. Obtain tutor approval in writing or migrate this layer before final submission.

3.5 Database and SQL

Six ordered SQL migrations create the current schema. UUID and bigserial keys identify durable entities and events. Enumerated types constrain difficulty, run status and obstacle kinds. Check constraints prevent negative balances and invalid levels; foreign keys enforce ownership; indexes support player history and leaderboard ordering; triggers maintain updated timestamps; and a view exposes leaderboard data without duplicating it. The API uses a PostgreSQL connection pool capped at ten clients and parameterised queries rather than interpolating user input.

The current model satisfies substantial persistence needs, but it is not yet the complete database described by the assignment. A future migration should add roles, permissions or a role column, administrator audit actions, player feedback, content-management entities where appropriate and report-friendly indexes. Run events should remain gameplay analytics; administrative actions require a separate immutable audit trail with actor, action, target, timestamp and before/after metadata.

3.6 Development methodology

An Agile iterative lifecycle is the best fit because movement feel, crowd behaviour, arithmetic readability and level difficulty cannot be validated reliably from documentation alone. The Agile Manifesto prioritises working software, collaboration and responding to change while retaining value

in processes and plans (Beck et al., 2001). The project uses those principles without claiming a complete formal Scrum implementation.

1. Discover: interpret the assignment, inspect comparable crowd runners, identify the learning problem and record constraints.
2. Design: write requirements, model data and flow, plan mobile wireframes, define level-generation and reward rules.
3. Build a vertical slice: home screen, one playable level, gates, crowd control, boss and result.
4. Expand: add database accounts, progression, skins, leaderboard, difficulty bands, obstacles and Android packaging.
5. Verify each iteration: automated rule tests, type checks, build checks, browser/device testing and database/API checks.
6. Review and adapt: interpret peer feedback, prioritise defects, update the SDD and preserve evidence in GitHub.

A practical two-week iteration can use a prioritised backlog, a defined acceptance condition, a short implementation period, automated checks, a device demonstration and a retrospective. Scrum's emphasis on inspection and adaptation supports this cadence, but roles and events should only be claimed if they were actually performed (Schwaber and Sutherland, 2020).

3.7 Version control and delivery

Git tracks source changes and GitHub hosts the MathRush3D repository. Commits should describe one coherent change, and feature branches or pull requests should be used for risky work. Tags should identify submitted versions. Secrets such as `DATABASE_URL` must remain in deployment environment variables and must not be committed. GitHub Actions can automate build and test workflows and retain APK artifacts; GitHub describes Actions as a way to execute customised software-development workflows, including CI/CD (GitHub, 2026).

- Minimum evidence for the final appendix: repository URL, commit history screenshot, branch/pull-request example, passing test/build workflow and tagged release.
- The Android pipeline must use a supported Node version, Java 21, Android SDK 36 and the repository's Gradle wrapper.
- Render environment variables must separately configure `DATABASE_URL`, database SSL, `NODE_ENV` and the exact comma-separated client origins.

3.8 Data protection, ethics and asset licensing

The service should collect only the data required for authentication, progression and competitive results. Email addresses and password hashes must never be exposed through leaderboard responses. Session records should expire and stale sessions should be removable. The final system should document a retention period for inactive accounts and provide an account-deletion process that relies on the schema's cascading relationships.

The game does not currently include chat, advertising, payments or location tracking, which reduces safeguarding and privacy risk. Display names still require validation because they are visible to other players. Arithmetic difficulty should be presented as a game setting rather than a judgement of ability, and feedback should avoid humiliating language when a player loses.

External models and textures must retain source and licence records. The current project uses local attribution metadata for Kenney and Quaternius assets. Any replacement asset should be accepted only after its redistribution and commercial-use terms are verified.

4. Testing and user acceptance

4.1 Testing strategy

Testing is layered so that fast deterministic checks catch rule regressions before slower device and user testing. Unit tests cover balance, crowd formation, level progression, speed progression and boss-meter results. TypeScript checks validate browser and server types. API integration tests should create a temporary account, authenticate, submit a run, retrieve progress and verify leaderboard ordering against a disposable database. End-to-end tests should then exercise registration, a complete level, pause/restart, next level and logout through the rendered interface.

4.2 Current automated evidence

Check	Command	Observed result	Date
Gameplay utility tests	npm test	5 test files passed; 30 of 30 tests passed in Vitest.	4 Aug 2026
Browser and server type checks	npm run typecheck	Completed successfully as part of the production build.	4 Aug 2026
Production build	npm run build	Completed successfully. Vite reported a 1,275.81 kB JavaScript bundle before gzip and issued a large-chunk warning.	4 Aug 2026
Database/API integration	Planned test script	No automated integration suite was identified in the inspected repository.	Pending
Android device matrix	Manual installation/UAT	Informal testing occurred, but a formal device matrix and signed evidence are still required.	Pending

4.3 Test data and environments

- Desktop web: current Chrome and Edge at 1366x768 and 1920x1080.
- Mobile web: 360x640, 390x844 and 412x915 responsive viewports, including on-screen keyboard behaviour.
- Android: at least one low/mid-range device and one newer device, recording OS version, memory and screen resolution.

- API/database: local PostgreSQL container and production Neon database with separate non-production test accounts.
- Adverse network: offline start, delayed first request, temporary API failure and retry after reconnection.

4.4 Peer tester plan

At least two peers should test independently. One tester should focus on first-time usability and mathematical clarity; the other should focus on mobile controls, difficulty and defects. The student should not coach testers beyond the instructions visible in the product. Record the tester's consent, device, browser/app version, date, observations, severity and suggested improvement. Do not replace observed evidence with assumed feedback.

Tester	Profile	Device/environment	Session	Evidence
[Tester name] 1	First-time player / learner	[Device, OS, browser]	Register, complete Easy 1-2, inspect result and leaderboard	Pending
[Tester name] 2	Mobile/gameplay tester	[Device, OS, APK version]	Test squeeze control, obstacles, pause/restart and boss/bonus sequence	Pending

4.5 User acceptance checklist

ID	Test action	Expected result	Result
UAT-01	Register with valid name, email and password.	Account is created and home screen loads.	Pending
UAT-02	Enter invalid or duplicate credentials.	Specific, readable validation/error response; no account corruption.	Pending
UAT-03	Reload and log in on another supported client.	Saved progression, coins, skin and statistics are restored.	Pending
UAT-04	Select an unlocked and a locked level.	Unlocked level starts; locked level cannot start and explains prerequisite.	Pending
UAT-	Play using touch drag and	Crowd follows input, remains in	Pending

ID	Test action	Expected result	Result
05	keyboard.	bounds and page does not scroll during play.	
UAT-06	Choose gates across all difficulties.	Expressions are legible, integer-valued and update the crowd correctly.	Pending
UAT-07	Press fully against each track wall near hazards.	Crowd visibly compresses and skilled positioning reduces casualties.	Pending
UAT-08	Collide with each obstacle type and an enemy crowd.	Correct individuals/damage are removed once; animation and sound match the event.	Pending
UAT-09	Reach the boss with an optimal route.	Level is beatable; timing meter is readable and boss damage is understandable.	Pending
UAT-10	Complete the bonus multiplier lane.	Crowd is consumed stage by stage, the track ends visibly and rewards match the result.	Pending
UAT-11	Pause, restart, replay, next level and home.	Each action changes state once and does not duplicate a run submission.	Pending
UAT-12	Disable sound/vibration and enable reduced effects.	Settings apply immediately and persist after restart/login.	Pending
UAT-13	Disconnect the network, finish a run, then reconnect.	Game remains playable; run queues and later synchronises once.	Pending
UAT-14	Use leaderboard difficulty filter/search function.	Results are correct and inputs cannot expose another player's private data.	Pending / feature gap
UAT-15	Attempt administrator functions as player/admin.	Player is denied; authorised administrator can use audited CRUD.	Blocked: not implemented

4.6 Performance evaluation plan

Performance claims should be supported by repeatable measurements rather than visual judgement. Each test must record the application version, device, operating system, browser or APK version, selected level, crowd size and reduced-effects setting.

Metric	Target	Method	Test point	Action if missed
Frame rate	At least 45 FPS on the selected mid-range Android device	Record a 60-second run with browser/Android profiling tools	Largest crowd, gate transition and boss battle	Reduce effects, profile allocations and review draw calls
Frame time	95th percentile below 22 ms	Capture performance trace and inspect long frames	Gate application and crowd casualties	Remove synchronous work and repeated object creation
Initial load	Playable screen within 5 seconds on normal 4G after service wake-up	Cold-cache timed test	First visit and first APK launch	Compress assets, split bundles and improve loading feedback
API latency	Warm health/profile request below 500 ms	Twenty production requests with median and 95th percentile	Login and post-run synchronisation	Inspect database queries, indexes and hosting region
Memory stability	No continuing growth after three complete runs	Compare memory snapshots before and after repeated runs	Replay and next-level sequence	Dispose scene resources and remove stale listeners
Offline recovery	One eventual run record with no duplicates	Finish offline, reconnect and query run history	Queued submission	Fix idempotency and retry handling

4.7 Defect and feedback handling

Each failed check should become a backlog item with reproduction steps, expected and actual behaviour, device/build information, severity and supporting image or video. Feedback should be interpreted rather than copied directly into a change: repeated observations, task failure and high-severity defects outrank individual visual preferences. After a fix, the original check and relevant

regression tests must be repeated. The final report should show at least one traceable chain from peer observation to decision, implementation, retest and evaluation.

5. Current assignment coverage and gaps

This matrix compares the inspected project and this report with the supplied brief. 'Partial' means that some evidence or functionality exists but the criterion should not yet be claimed as achieved.

Criterion	Current evidence	Assessment	Next evidence/action
P1 Problem and requirements	Problem definition, stakeholders, user/system requirements and diagrams in this report.	Substantial draft	Validate requirements with named users and update after feedback.
P2 Risks	Risk register covers gameplay, performance, security, data, licensing, mobile and compliance.	Substantial draft	Add owner review dates and evidence of mitigations.
M1 SDD analysis	SDD connects the learning problem to architecture, data and game flow.	Partial	Add requirement traceability to implementation screenshots/commits.
P3 Tools and techniques	Selected stack, purpose and official research references documented.	Substantial draft	Add development screenshots and actual usage examples.
M2 Justification	Alternatives compared across 3D, state, backend, data, mobile and methodology.	Partial	Deepen comparison using measured build/performance evidence.
D1 Evaluation of solution/methodology	Only preliminary evaluation is possible at this stage.	Pending	Evaluate after UAT and compare outcomes against alternatives.
P4 Formal peer presentation	Tester plan exists; no presentation evidence supplied.	Pending	Deliver presentation and attach date, audience, slides and feedback.
P5 Functional	Playable web/APK, README, database notes and this revised report exist.	Partial	Add user/admin guide, screenshots and backend compliance resolution.

Criterion	Current evidence	Assessment	Next evidence/action
app/support docs			
M3 Interpret feedback	Feedback method is defined but no completed peer results exist.	Pending	Complete UAT, group themes and justify accepted/rejected changes.
M4 Develop against SDD	Modular implementation and passing rule/type checks exist.	Partial	Map requirements to commits, tests and screenshots.
P6 Review performance	Performance design choices are described.	Pending/partial	Capture FPS, load time, bundle size and device results.
M5 Critical review of stages/risks	Initial risks and lifecycle are documented.	Pending/partial	Compare estimated and actual risks after remaining iterations.
D2 Improve and future work	Open gaps are prioritised below but not yet feedback-validated.	Pending	Link final recommendations to test and peer evidence.

5.1 Mandatory feature gap analysis

Brief requirement	Current state	Required action
PostgreSQL and SQL	Implemented through migrations, parameterised queries, constraints and a leaderboard view.	Add schema/query evidence and backup/restore test.
Frontend React + CSS framework	Implemented with React, Tailwind CSS and custom CSS.	Add responsive screenshots and accessibility evidence.
Backend PHP/Python/Java	Core backend currently uses Node.js/Express/TypeScript.	Obtain explicit approval or migrate to FastAPI, Spring

Brief requirement	Current state	Required action
		Boot or PHP before submission.
CRUD	Account/profile, progress, skins and run creation/read paths exist; complete admin CRUD is absent.	Define core entities and implement protected create/read/update/delete workflows.
RBAC	No role/permission model or admin interface identified.	Add roles, authorisation middleware, admin UI and denial tests.
Validation/error handling	Zod, database constraints and central API error handling exist.	Add integration tests and user-friendly error mapping.
Search/filter	Leaderboard supports limited retrieval/filter behaviour; general search is incomplete.	Add documented server-side filters/search with pagination.
Dashboard/report	Player statistics and leaderboard exist.	Add administrator reporting and exportable summary if required.
Responsive UI	Mobile styles, safe areas, scroll fixes and touch controls exist.	Complete formal viewport/device matrix.
Audit logs	Gate and obstacle events are analytics, not administrator audit records.	Create separate audit_log entity and record protected changes.
Feedback	No persistent feedback entity or completed peer evidence identified.	Add feedback table/form if required and complete UAT records.

5.2 Prioritised next stage

7. Resolve the backend-language compliance decision with the tutor before investing further in the existing service.
8. Implement RBAC, administrator authentication/authorisation, protected CRUD, audit logs and denial tests.
9. Add API/database integration tests, server-side reward plausibility checks and repeatable test data.

10. Complete search/filter, administrator reports and a persistent feedback workflow where the full guideline requires them.
11. Run peer UAT on web and Android, interpret results, implement justified changes and retest.
12. Capture final evidence: diagrams, SQL, commits, CI, responsive screens, performance metrics and versioned deployment URLs.

6. Conclusion

Math Rush 3D already demonstrates a coherent, technically ambitious application: a React-based 3D game, a persistent PostgreSQL data model, secure account handling, responsive Android delivery and automated rule checks. Its strongest design decision is the use of a single generated level definition and shared reward rules to reduce disagreement between gameplay systems and stored outcomes. The clearest weaknesses are academic compliance around the backend language, missing role-based administration and the absence of completed peer/evaluation evidence.

This report covers the topics listed in the supplied project guideline: executive summary, SDD, problem and stakeholder analysis, requirements, measurable success criteria, risk review, use-case, ER, architecture, DFD, flowchart and wireframe designs, requirement traceability, software choices and alternatives, frontend, backend and database discussion, methodology, version control, ethics, performance evaluation and a peer UAT plan. The report does not claim completion of evidence that has not been supplied. Peer testing, measured device performance, administration functions and the backend-language decision remain necessary before the related higher-grade criteria can be evidenced confidently.

References

Beck, K. et al. (2001) Manifesto for Agile Software Development. Available at: [Agile Manifesto](#) (Accessed: 21 July 2026).

GitHub (2026) GitHub Actions documentation. Available at: [GitHub Actions documentation](#) (Accessed: 21 July 2026).

Ionic (2026) Capacitor: Cross-platform native runtime for web apps. Available at: [Capacitor documentation](#) (Accessed: 21 July 2026).

Meta Platforms (2026) React Quick Start. Available at: [React Quick Start](#) (Accessed: 21 July 2026).

Microsoft (2026) TypeScript documentation. Available at: [TypeScript documentation](#) (Accessed: 21 July 2026).

OpenJS Foundation (2026) Express 5: Installing and TypeScript. Available at: [Express documentation](#) (Accessed: 21 July 2026).

OWASP Foundation (2026a) Password Storage Cheat Sheet. Available at: [OWASP Password Storage Cheat Sheet](#) (Accessed: 21 July 2026).

OWASP Foundation (2026b) Session Management Cheat Sheet. Available at: [OWASP Session Management Cheat Sheet](#) (Accessed: 21 July 2026).

Poimandres (2026) React Three Fiber: Introduction. Available at: [React Three Fiber introduction](#) (Accessed: 21 July 2026).

PostgreSQL Global Development Group (2026) What is PostgreSQL? PostgreSQL 18 Documentation. Available at: [PostgreSQL documentation](#) (Accessed: 21 July 2026).

Schwaber, K. and Sutherland, J. (2020) The Scrum Guide. Available at: [The Scrum Guide](#) (Accessed: 21 July 2026).

Tailwind Labs (2026) Styling with utility classes. Available at: [Tailwind CSS documentation](#) (Accessed: 21 July 2026).

Three.js Authors (2026) Three.js manual. Available at: [Three.js manual](#) (Accessed: 21 July 2026).

Vitest (2026) Getting Started. Available at: [Vitest guide](#) (Accessed: 21 July 2026).

Vite (2026) Getting Started. Available at: [Vite guide](#) (Accessed: 21 July 2026).

Appendix A. Evidence still to insert

- The remaining tutor guideline and any revised section order.
- Figma project link or exported annotated wireframes if specifically required.
- Dated screenshots of registration, home/level select, gameplay, boss meter, bonus finish, results, skins, statistics and leaderboard.
- GitHub repository, commit history, pull request/branch and passing CI evidence.
- Database migration and representative SQL query evidence with sensitive values redacted.
- Named peer presentation and UAT evidence, feedback interpretation, implemented changes and retest results.
- Measured performance results on the specified desktop and Android device matrix.
- Written tutor decision on the Node.js/Express backend or evidence of migration to an accepted language.

Appendix B. Project technology snapshot

Layer	Current implementation
Web client	React 19.2, React DOM, TypeScript 6, Vite 8, Tailwind CSS 4, HTML/CSS
3D/game	Three.js 0.185, React Three Fiber 9.6, Drei 10.7, WebGL, GLB/GLTF, GPU instancing
State/audio	Zustand 5 with persistence; Web Audio API procedural effects
API/security	Node.js, Express 5.2, Zod 4, Helmet, CORS, rate limiting, bcryptjs, crypto session tokens
Data	PostgreSQL, pg connection pool, six SQL migrations, Neon production, Docker local development
Android	Capacitor 8.4, Java/AndroidX, Gradle, min SDK 24, target/compile SDK 36, immersive fullscreen
Quality	Vitest, TypeScript, Oxlint, Git/GitHub, GitHub Actions
Hosting	Render static web client, Render API service, Neon PostgreSQL
Assets	CC0 Kenney and Quaternius GLB assets with local attribution metadata